Development of new telecommunications services using an expert system

A Eberlein, M Crowther and F Halsall

This paper outlines the functionality of an expert system currently under development. The 'Requirements Assistant for Telecommunications Services' (RATS) is designed to actively assist the development of new telecommunications services. It helps during the early parts of the development life cycle, during requirements capture and analysis, leading to formal specifications of the service at different stages of refinement. Facilities for requirements traceability, rationale and documentation as well as impact analysis of requirements change are provided. The tool contains comprehensive knowledge bases with the main characteristics of the telecommunications domain and the development process of telecommunications services, which enable the tool to actively guide the software developer. Assistance is provided for three dimensions of the development process of functional requirements — refinement, formality and completeness. The output of the RATS tool enables a smooth transition to the ITU-recommended Specification and Description Language (SDL), which allows automated analysis of static and dynamic properties as well as code generation.

1. Introduction

Studies [1, 2] indicate that errors associated with requirements are the most numerous and, more significantly, they are the most costly and time-consuming to correct. This has made requirements engineering (RE) a discipline of increasing importance. Due to the immense amount of information involved in RE for software projects, good tool support is essential. It has been pointed out [3] that current commercial CASE tools are not adequate for RE and are therefore not used in RE work. Even commercial RE tools, which are mostly able to provide basic requirements management functions, cannot provide domain-specific guidance and lack the capability of decision tracking.

These problems have become apparent in the telecommunications domain which, from a software engineering viewpoint, can be considered as a complex distributed computer system. It is estimated that at least 70% of the design effort associated with telecommunications is concerned with the software engineering domain [4]. A lot has already been invested in order to increase the quality of telecommunications software. One of the major progressions was the adoption of formal methods. The International Telecommunications Union (ITU) recommended the use of the Specification and Description Language (SDL) [5] as a formal, executable language to specify services and protocols. There is now good commercial tool support available which allows analysis, verification, execution and validation of SDL specifications as well as automatic code and test case generation.

The major bottle-neck in the cycle is currently found in the process of requirements elicitation and early analysis in order to get a high-quality requirements specification. The 'Requirements Assistant for Telecommunications Services' (RATS) tool is a prototype providing a front-end to currently practised development methods. It actively helps during requirements elicitation and early analysis while providing a smooth transition to SDL-based formal methods, overcoming the inadequacy of commercial RE tools. The knowledge-based expert system, implemented with the help of a knowledge-representation language (in this case TELOS [6]), provides a feasible solution to today's problems of RE.

2. Overall methodology for the design of new IN services

The development methodology is shown in Fig 1. It uses the RATS tool and can be inserted into already existing life cycles which use formal methods. It provides a semi-formal step between the informal service idea and the formal specification in a specification language (e.g. LOTOS, SDL). This means that the input of the RATS tool is the service idea expressed in customer terms, and the output is in a semiformal form easily translatable into SDL.

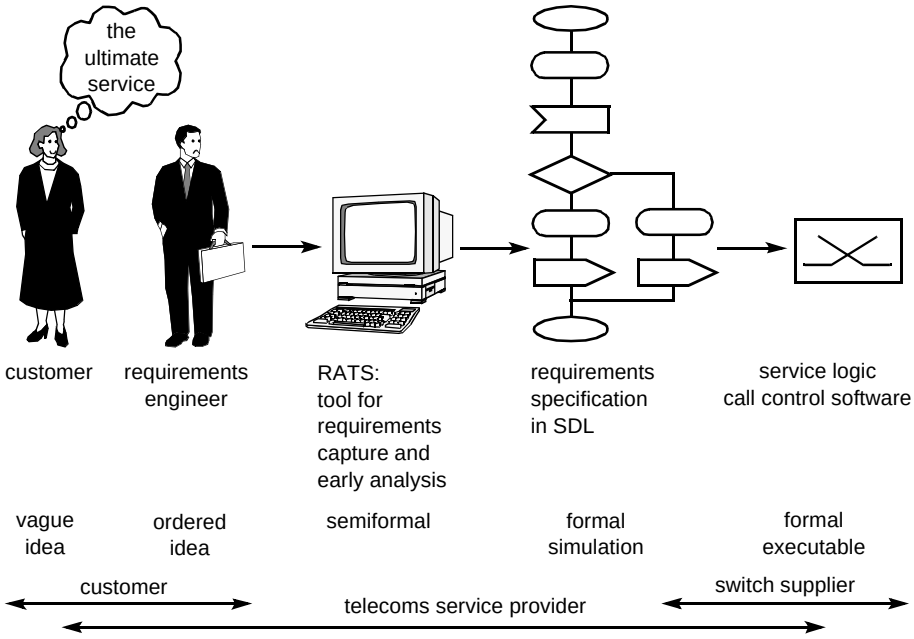


Fig 1 The overall methodology for service design.

3. Outline of the RATS tool architecture

The increasing complexity and diversity of the telecommunications domain make it difficult for a service developer to have detailed expertise of all the factors to be considered during service design. It therefore seems feasible to use a knowledge-based approach in order to alleviate the service designer from knowing all the detailed facts of the telecommunications domain. In general, the RATS tool (see Fig 2) comprises three parts:

- the user interface,
- information about the development processes,
- the main characteristics of the telecommunications domain.

However, this approach has some disadvantages — changes in the higher layers of the conceptual model involve considerable effort. Fortunately, the nature of the telecommunications domain changes relatively slowly due

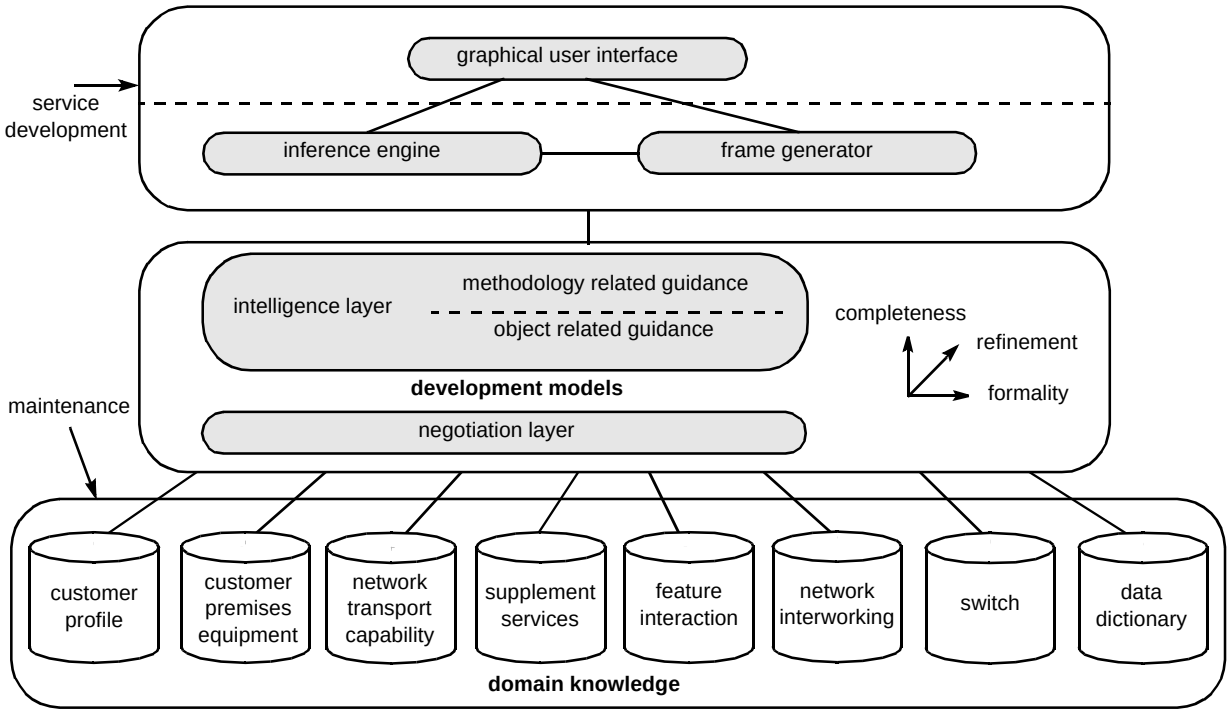


Fig 2 The RATS tool.

to the costs involved in major upgrades. Another problem is the considerable amount of information necessary to achieve active assistance during service development which results in large knowledge bases. Already the prototype, which does not yet contain all the intended information, has clearly observable response times which are likely to further increase with the completion of the tool.

3.1 Domain models

The domain models contain the main characteristics of telecommunications-specific issues. Currently, the RATS prototype contains information from more than 80 recommendations and standards conceptually expressed in TELOS.

Due to a clear separation of the development and domain models, a great variety of models can be used within RATS. Very detailed models, which can be seen as electronic versions of standards and which are suitable for teaching purposes, can be used. Other models might just contain the absolutely necessary information to support the service design process. This shows that the RATS tool can accommodate very different kinds of domain model which might have been created by different people for different purposes. This is possible by introducing a negotiation layer (see section 3.3) between the domain models and the development models.

3.2 Development models

The development models describe a development process suitable for requirements acquisition and early analysis of telecommunications services. The basic outline of the initial development process is shown in Fig 3.

The foundation of the model is a sophisticated subclass hierarchy which ensures a development along three dimensions — completeness, refinement and formality. The process attempts to leave as much freedom as possible in the beginning in order to make sure that the end product is really what the customer wants, but then formality is stepwise introduced. This starts with an initial assignment of requirements to a requirements subclass which then channels the further development. The further down the subclass hierarchy the designer goes, the less freedom there is. In the case of functional requirements, the subclass hierarchy leads to a use-case design process [7]. The most formal use-case definition consists of a semiformal use-case syntax easily translatable into SDL.

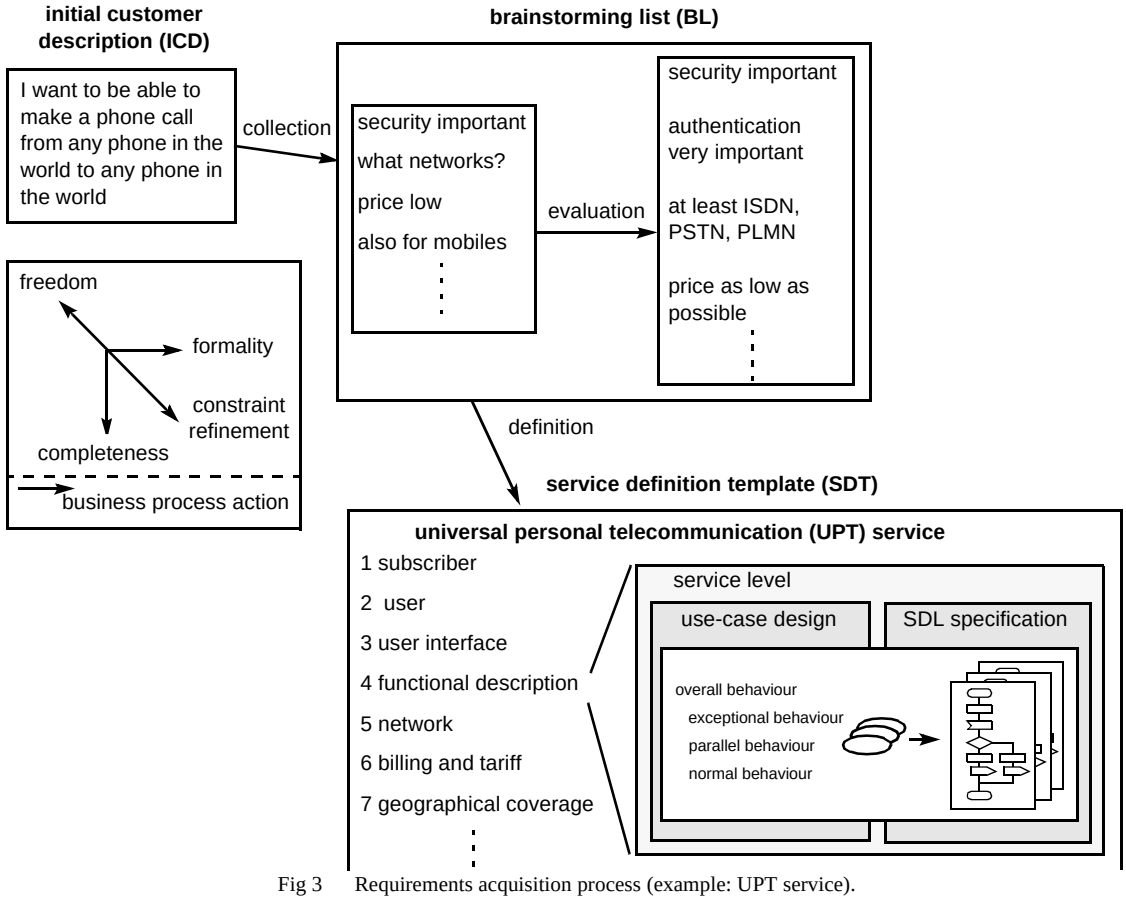


Fig 3 Requirements acquisition process (example: UPT service).

3.3 The negotiation layer

In order to enable the use of different kinds of domain models (see section 3.1) and to offer active, domain-specific guidance, there needs to be an interface between the development models and the domain models. Intermodel consistency could be another task of this layer; however, RATS provides this in the intelligence layer (see section 3.4).

Despite informal negotiation being very much part of everyday life, negotiation behaviour during requirements specification has not yet been explored very well [8]. In this case, a relatively simple approach is taken.

- **Negotiation objects** — these objects form a subclass of the class Requirement. They therefore inherit all characteristics of this class but fulfil a special function in the chain of requirements development. From a requirements management viewpoint they are the 'end' of a chain of requirements, i.e. they link to an object in one of the domain models. These negotiation objects play a special role as they have to perform the connection between the development process and the different kinds of domain models thus allowing a clear separation between the models.
- **Keywords** — the negotiation process itself is part of the problem of software reuse [9]. In this case, keyword searches of different categories are employed supported by two kinds of meta-attributes which can reuse already existing domain objects, or allow new keywords to be inserted.

As the active support of the negotiation process is done by a string comparison between the desired keywords provided by the service designer and the keywords assigned to existing domain features, there is one major impact on the domain models with which they have to comply. All keywords need to be positive descriptions of the object as negations will lead to erroneous results. This principle can be termed as 'positive modelling'.

The combination of the functionality of the negotiation layer and the intelligence layer (see section 3.4) is a means for active assistance for software reuse. The domain models contain specification and software modules of generic network functionality available for reuse. This is one of the main motivations behind the intelligent network concept [10].

3.4 The intelligence layer

This section is dedicated to the question of how intelligence can be provided in a system implemented in TELOS.

Passive guidance

Passive guidance can be provided via several methods.

- Conceptual, object-oriented modelling using formal languages provides a certain level of intelligence. Inheritance, classification, specialisation and instantiation are means for some basic forms of intelligence. TELOS and its implementation in the ConceptBase tool [11, 12] contains several axioms which are automatically enforced as soon as an object is inserted into the knowledge base.
- In addition to these built-in rules and constraints, the created domain models contain a large number of permanent, user-defined constraints and rules which ensure model consistency. Furthermore, the development models as well as the negotiation layer contain several constraints and rules which help in the correct development of services.

The two features mentioned above prevent the insertion of inconsistent objects into the knowledge base. However, this is only wanted for serious consistency violations. Some inconsistencies (especially incompleteness) can be temporarily acceptable [13]. To accommodate this, two additional solutions have been provided.

- A set of user-defined constraints is available, but these are only triggered at certain milestones of the development life cycle in order to check consistency and completeness.
- The RATS tool contains, at a high layer of the models, generic meta-rules which assign an object to certain 'state classes' showing the state an object is currently in with respect to the development process.

All the four methods are basic but crucial, and provide passive guidance ('Don't do that!'). However, what is expected of an expert system is the additional feature of active guidance ('Do this!').

Active guidance

There are again several methods by which active guidance can be achieved using TELOS. Some of the possibilities are briefly outlined here.

- The classes of the development models contain strings which point out the steps which normally have to be performed. However, this allows only general guidance as it is connected to a class and not to the current state of a specific instance.
- Some rough guidance can be derived from the development models, which must be oriented towards the development process rather than the conceptual

inheritance of characteristics. Although in many cases these two aspects might be the same, this cannot generally be assumed.

- Special attributes are created which point to the object to be dealt with next. This is similar to the point above, except that it leaves more freedom to the conceptual model.

- The most general approach for active guidance involves the creation of special intelligence models. Rules ensure the assignment of specific intelligence objects to instances of any requirements object depending on the actual state of this specific instance. Complex rules have to be created in order to allow such individual, active guidance.

- Queries for possible parameters for the intelligence objects can be defined.

Artificial intelligence in RATS

RATS employs a combination of several of the above mentioned ideas. In order to illustrate the approach used, a simple example is given.

Assume the UPT (universal personal telecommunication) service is to be defined. The first method of passive guidance ensures that the definition is an instance of the class ServiceDefinitionTemplate (which is a subclass of RequirementsDocument) in order to enable the inheritance of its attributes. This document may still contain some requirements which have not yet got the status agreed. The RATS tool's generic meta-rules inform the designer that there is still something wrong with this document and hence the document is assigned to the class InconsistentObject. Active guidance provides an intelligence object which points out to the designer that efforts should be made to get the status agreed for all requirements in this requirements document. Then a parameter query points out which requirements still need to get the status agreed. In case the designer ignores these messages and tries to assign the status agreed to the whole document (even if it still contains requirements without the status agreed), the permanent, user-defined constraints will reject this attempt.

Another task of the intelligence layer is to ensure intermodel consistency. Rules are used to guarantee meaningful usage of the libraries contained in the independently created domain models, i.e. that the elements used from different domain libraries fit together. The following few lines show a rule ensuring that only such a basic network service can be specified in a service definition which is also offered by one of the chosen networks:

```
NetworkAndBasicServicesFitTogether_rule : $forall f1/ServiceDefi-
nitionTemplate
  (exists e1,e5/NegotiationObject e2/NW e4/Library
    (f1 containsNetwork e1) and (f1 containsBasicServices e5) and
     (e1 linksToLibrary e2) and (e5 linksToLibrary e4) and
   not(exists e3/BasicNWService (e2 offers e3) ==> (e4 in e3)))
  ==> (f1 IntelligenceObject Int25)$
```

At the moment, the RATS tool contains altogether some 290 user-defined rules and 63 user-defined constraints.

3.5 Requirements management

Good requirements management is essential in RE. RATS uses a novel, uniform treatment of very different kinds of requirements which goes beyond the capabilities of currently available RE tools. It allows a high degree of flexibility and ease of management at all levels of abstractions. Depending on the kind of requirement and the operation performed on it, default attributes and links are automatically created. The historical development and the rationale behind decisions is recorded, which is necessary for requirements traceability. In a forward direction, impact analysis of changing requirements can be assessed by following logical dependency links between requirements.

4. Conclusions and future work

The RATS tool is intended as a step towards a partial solution of the current problems of tool support for RE. Its strength is that it fits into already existing development life cycles which employ formal methods. Its main novelties are:

- requirements are captured freely from customers and a best-fit with existing infrastructure is achieved by negotiation mediated by the RATS tool, which uses knowledge about telecommunications networks and services to actively support this process,
- requirements are managed uniformly and in electronic format, regardless of levels of abstraction, allowing much improved standards of traceability over current commercial tools,
- an intermediate formal description of the service is developed, using the specification language SDL, in advance of implementation, thus encouraging complete and unambiguous specifications and allowing early validation activities, such as service simulation and feature interaction analysis.

The prototype of the RATS tool still needs further development and this will influence the already existing models. Up to now, only small parts of the UPT service have been used to demonstrate the usefulness of the tool and a more complete case study needs to be developed. This will also help to make more obvious any possible performance problems.

References

1 Boehm B: 'Verifying and validating software requirements and design specifications', IEEE Software, 1 No 1, pp 94—103 (1984).

2 Davis A M: 'Software requirements — objects, functions and states', Prentice Hall (1993).

3 Bubenko J: 'Challenges in requirements engineering', Second IEEE International Symposium on Requirements Engineering (March 1995).

4 Reed R et al: 'Methods for service software design', IEE (1992).

5 ITU-T Recommendation Z.100: 'CCITT Specification and Description Language (SDL)', (June 1994)

6 Mylopoulos J et al: 'TELOS: representing knowledge about information systems', ACM Transactions on Information Systems, 8, No 4, pp 325—362 (1990).

7 Jacobson I: 'Object-oriented software engineering — a use-case driven approach', Addison-Wesley (1993).

8 Robinson W: 'Negotiation behaviour during requirements specification', IEEE (1990).

9 Bellinzona R et al: 'Reuse of specifications and designs in a development information system', IFIP Working Conference on Information System Devevlopment Process, IFIP WG 8.1 (September 1993).

10 Garrahan J et al: 'Intelligent network overview', IEEE Comunications Magazine (March 1993).

11 Jarke M: 'ConceptBase — a deductive object base manager', University of Aachen, Germany (1993).

12 Jarke M: 'ConceptBase V4.1 user manual', University of Aachen, Germany (1996).

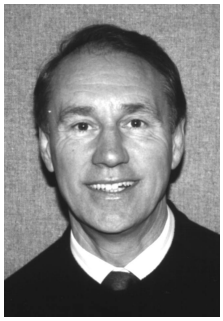
13 Finkelstein A et al: 'Inconsistency handling in multi-perspective specifications', IEEE Transactions on Software Engineering, 20, No 8, pp 569—578 (August 1994).

.

Armin Eberlein graduated from the 'Fachhochschule fuer Technik' in Mannheim, Germany in 1993. He then spent some time working as hardware and software developer in Siemens in Munich, Germany. This was followed by a postgraduate course in Communication Systems at the University of Wales, Swansea, UK, leading to a Master's degree. He is currently studying for a PhD also in Swansea. His research is sponsored by BT and concerned with formal methods in system design, AI in telecommunications and new methodologies for service design.

Michael Crowther holds degrees in Electrical and Electronic Engineering from the University of Leeds and in Telecommunications Technology from the University of Aston. He worked for GPT on digital hardware developments on the UXD5 and System X switches for 4 years before joining BT in 1986. In BT he has contributed to numerous signalling and network intelligence developments including specification of Signalling System No. 7 and implementation of DPNSS interfaces. Many of these recent developments have involved the introduction of formal specification and validation techniques through the use of the ITU's specification language SDL. He is currently chairman of the BT SDL user group.

He is a member of the IEE and a Chartered Engineer.



Fred Halsall is Professor of Communications Engineering at the University of Wales, Swansea and head of the Communications Research Group.

The Group is involved in research in computer network modelling, software engineering, and high bit rate data transmission over radio. He has published widely in these and related areas.

He is a Fellow of the IEE.